

Demonstração do Controle de Concorrência

Foi realizado um teste utilizando duas sessões simultâneas do PostgreSQL para simular duas transações tentando escalar o mesmo residente para o mesmo dia, turno e unidade.

Sessão 1 – Lock Pessimista (Terminal 1)

```
eloizamonteiro@eloizamonteiro-IdeaPad-Slim-3-15ARP10:~/Sistema-de-Gestao-Hospitalar$ sudo docker exec -it postgres psql -U postgres -d sgh_db
psql (16.14)
Type "help" for help.

sgh_db=# \timing on
Timing is on.
sgh_db=# BEGIN;
BEGIN
Time: 0.277 ms
sgh_db=#
sgh_db=# SELECT id_profissional
sgh_db=# FROM residente
sgh_db=# WHERE id_profissional = 8
sgh_db=# FOR UPDATE;
 id_profissional
-----
            8
(1 row)

Time: 2.773 ms
sgh_db=# INSERT INTO escala (
sgh_db=#     data_plantao,
sgh_db=#     turno,
sgh_db=#     id_unidade,
sgh_db=#     id_residente,
sgh_db=#     id_preceptor
sgh_db=# )
sgh_db=# VALUES (
sgh_db=#     '2026-09-01',
sgh_db=#     'MANHA',
sgh_db=#     1,
sgh_db=#     8,
sgh_db=#     13
sgh_db=# );
INSERT 0 1
Time: 9.458 ms
sgh_db=#
```

A primeira transação utiliza SELECT ... FOR UPDATE para aplicar um lock pessimista sobre o registro do residente. Em seguida, insere a escala e mantém a transação aberta, impedindo que outra transação modifique o mesmo registro até que seja executado o comando COMMIT.

Sessão 2 – Transação Bloqueada (Terminal 2)

```
eloizamonteiro@eloizamonteiro-IdeaPad-Sli
eloizamonteiro@eloizamonteiro-IdeaPad-Slim-3-15ARP10:~/Sistema-de-Gestao-Hospitalar$ sudo docker
exec -it postgres psql -U postgres -d sgh_db
[sudo] senha para eloizamonteiro:
psql (16.14)
Type "help" for help.

sgh_db=# \timing on
Timing is on.
sgh_db=# BEGIN;
BEGIN
Time: 0.626 ms
sgh_db=*#
sgh_db=*# SELECT id_profissional
sgh_db=*# FROM residente
sgh_db=*# WHERE id_profissional = 8
sgh_db=*# FOR UPDATE;
```

A segunda transação tenta acessar o mesmo registro utilizando SELECT ... FOR UPDATE. Como o registro já está bloqueado pela primeira transação, ela permanece aguardando a liberação do lock até que o COMMIT seja executado na primeira sessão.

Tentativa de Inserção Duplicada

Terminal 1 Terminal 2

```

eloizamonteiro@eloizamonteiro-IdeaPad-Slim-3-15ARP10:~/Sistema-de-Gestao-Hospitalar$ sudo docker exec -it postgres psql -U postgres -d sgh_db
[sudo] senha para eloizamonteiro:
psql (16.14)
Type "help" for help.

sgh_db=# \timing on
Timing is on.
sgh_db=# BEGIN;
BEGIN
Time: 0.277 ms
sgh_db=#
sgh_db=# SELECT id profissional
sgh_db=# FROM residente
sgh_db=# WHERE id profissional = 8
sgh_db=# FOR UPDATE;
 id profissional
-----
            8
(1 row)

Time: 2.773 ms
sgh_db=# INSERT INTO escala (
sgh_db=#   data_plantao,
sgh_db=#   turno,
sgh_db=#   id_unidade,
sgh_db=#   id_residente,
sgh_db=#   id_preceptor
sgh_db=# )
sgh_db=# VALUES (
sgh_db=#   '2026-09-01',
sgh_db=#   'MANHA',
sgh_db=#   1,
sgh_db=#   8,
sgh_db=#   13
sgh_db=# );
INSERT 0 1
Time: 9.458 ms
sgh_db=# COMMIT;
COMMIT
Time: 5.427 ms
sgh_db=#

```

```

eloizamonteiro@eloizamonteiro-IdeaPad-Slim-3-15ARP10:~/Sistema-de-Gestao-Hospitalar$ sudo docker exec -it postgres psql -U postgres -d sgh_db
[sudo] senha para eloizamonteiro:
psql (16.14)
Type "help" for help.

sgh_db=# \timing on
Timing is on.
sgh_db=# BEGIN;
BEGIN
Time: 0.626 ms
sgh_db=#
sgh_db=# SELECT id profissional
sgh_db=# FROM residente
sgh_db=# WHERE id profissional = 8
sgh_db=# FOR UPDATE;
 id profissional
-----
            8
(1 row)

Time: 73195.457 ms (01:13.195)
sgh_db=# INSERT INTO escala (
sgh_db=#   data_plantao,
sgh_db=#   turno,
sgh_db=#   id_unidade,
sgh_db=#   id_residente,
sgh_db=#   id_preceptor
sgh_db=# )
sgh_db=# VALUES (
sgh_db=#   '2026-09-01',
sgh_db=#   'MANHA',
sgh_db=#   1,
sgh_db=#   8,
sgh_db=#   13
sgh_db=# );
ERROR:  0 residente já possui uma escala neste dia e turno.
CONTEXT:  PL/pgSQL function fn_check_sobreposicao_escala() line 13 at RAISE
Time: 7.001 ms
sgh_db=#

```

Após a liberação do lock, a segunda transação tenta inserir uma escala idêntica. A trigger `trg_check_sobreposicao_escala` identifica a duplicidade e impede a inserção, preservando a integridade dos dados e evitando inconsistências na tabela escala.

Resultado Final

Terminal 1 Terminal 2

```

eloizmonteiro@eloizmonteiro-IdeaPad-Slim-3-15ARP10: ~/Sistema-de-Gestao-Hospitalar$
sudo docker exec -it postgres psql -U postgres -d sgh_db
[sudo] senha para eloizmonteiro:
psql (16.14)
Type "help" for help.

sgh_db=# \timing on
Timing is on.
sgh_db=# BEGIN;
BEGIN
Time: 0.277 ms
sgh_db=#
sgh_db=# SELECT id_profissional
sgh_db=# FROM residente
sgh_db=# WHERE id_profissional = 8
sgh_db=# FOR UPDATE;
 id_profissional
-----
            8
(1 row)

Time: 2.773 ms
sgh_db=# INSERT INTO escala (
sgh_db=#     data_plantao,
sgh_db=#     turno,
sgh_db=#     id_unidade,
sgh_db=#     id_residente,
sgh_db=#     id_preceptor
sgh_db=# )
sgh_db=# VALUES (
sgh_db=#     '2026-09-01',
sgh_db=#     'MANHA',
sgh_db=#     1,
sgh_db=#     8,
sgh_db=#     13
sgh_db=# );
INSERT 0 1
Time: 9.458 ms
sgh_db=# COMMIT;
COMMIT
Time: 5.427 ms
sgh_db=# SELECT COUNT(*) AS quantidade_escalas
sgh_db=# FROM escala
sgh_db=# WHERE id_residente = 8
sgh_db=# AND data_plantao = '2026-09-01'
sgh_db=# AND turno = 'MANHA';
 quantidade_escalas
-----
                1
(1 row)

Time: 1.010 ms
sgh_db=#

```

```

eloizmonteiro@eloizmonteiro-IdeaPad-Slim-3-15ARP10: ~/Sistema-de-Gestao-Hospitalar$ sudo docker
exec -it postgres psql -U postgres -d sgh_db
[sudo] senha para eloizmonteiro:
psql (16.14)
Type "help" for help.

sgh_db=# \timing on
Timing is on.
sgh_db=# BEGIN;
BEGIN
Time: 0.626 ms
sgh_db=#
sgh_db=# SELECT id_profissional
sgh_db=# FROM residente
sgh_db=# WHERE id_profissional = 8
sgh_db=# FOR UPDATE;
 id_profissional
-----
            8
(1 row)

Time: 73195.457 ms (01:13.195)
sgh_db=# INSERT INTO escala (
sgh_db=#     data_plantao,
sgh_db=#     turno,
sgh_db=#     id_unidade,
sgh_db=#     id_residente,
sgh_db=#     id_preceptor
sgh_db=# )
sgh_db=# VALUES (
sgh_db=#     '2026-09-01',
sgh_db=#     'MANHA',
sgh_db=#     1,
sgh_db=#     8,
sgh_db=#     13
sgh_db=# );
ERROR:  0 residente já possui uma escala neste dia e turno.
CONTEXT:  PL/pgSQL function fn_check_sobreposicao_escala() line 13 at RAISE
Time: 7.001 ms
sgh_db=# ROLLBACK;
ROLLBACK
Time: 0.258 ms
sgh_db=#

```

A consulta final confirma que apenas uma escala foi registrada para o residente na mesma data, turno e unidade.

Dessa forma, o teste demonstra a utilização do lock pessimista para controle de concorrência e da trigger `trg_check_sobreposicao_escala` para garantir a integridade dos dados, impedindo que duas transações registrem escalas conflitantes para o mesmo residente.